

Analyzing the Intersection of Dementia Trends and Healthcare Resources

Dementia and Healthcare Resource Analysis

Introduction:

Data science has become increasingly instrumental in transforming dementia care and healthcare resource management by enabling the collection and analysis of diverse datasets including cognitive assessments, biomarkers, healthcare facility distributions, and workforce demographics. These comprehensive data sources provide healthcare providers with crucial insights for developing targeted interventions and resource allocation strategies. As data science continues to evolve within healthcare systems, it creates unprecedented opportunities for understanding disease progression patterns and optimizing healthcare delivery for neurodegenerative conditions. In alignment with this data-driven approach to healthcare, this research aims to analyze the intersection between dementia trends and healthcare resources, exploring the relationships between cognitive decline patterns, biomarker profiles, and the evolution of healthcare infrastructure and workforce capacity. By examining these relationships, we seek to identify patterns that can inform more effective strategies for dementia care delivery and resource allocation.

Aim

This project aims to analyze the relationship between dementia incidence trends and healthcare resources, identifying patterns and actionable insights.

Data

Data Sources Utilised:

For this project, five datasets were chosen carefully to conduct a comprehensive research about this project. All data were stored in CSV files that contain diverse range of information which provided a strong foundation for detailed analysis.

The following are the datasets used in this project:

1. Dementia datasets (Source: <https://github.com/kaushiika/Healthcare-Data-Analysis/blob/main/Dementia.csv> (<https://github.com/kaushiika/Healthcare-Data-Analysis/blob/main/Dementia.csv>))
2. Plasma lipidomics in Alzheimers disease and its progression ([https://www.kaggle.com/datasets/fereshtehjozaghar/plasma-](https://www.kaggle.com/datasets/fereshtehjozaghar/plasma-(https://www.kaggle.com/datasets/fereshtehjozaghar/plasma-)) (<https://www.kaggle.com/datasets/fereshtehjozaghar/plasma->) lipidomics-in-alzheimers-disease/data)
3. Community-Based Dementia Care (Source: <https://catalog.data.gov/dataset/community-based-dementia-care> (<https://catalog.data.gov/dataset/community-based-dementia-care>))
4. Health Facilities and Beds in Inpatient Facilities (Source: <https://beta.data.gov.sg/collections/521/view> (<https://beta.data.gov.sg/collections/521/view>))
5. Healthcare Professional to Population Ratio (Source: <https://beta.data.gov.sg/collections/490/view> (<https://beta.data.gov.sg/collections/490/view>))

Data Cleaning and Analysis:

The following section delves into all the datasets utilized in this project. It begins with an examination of the initial dataset, followed by steps such as formatting the dataset for clarity and ease of graph plotting. Additionally, the data cleaning process will be carried out to check for missing values and identify potential outliers. Furthermore, a general analysis will be conducted to provide a descriptive statistical summary of each dataset.

First dataset, Dementia datasets

Dementia, a debilitating neurological condition characterized by cognitive decline, affects millions worldwide. Alzheimer's disease (AD) is the most common form of dementia, accounting for 60-80% of cases. Early detection and monitoring of dementia progression are crucial for effective treatment and management. This necessitates the development of reliable clinical biomarkers that can identify individuals at risk and track disease advancement. This discussion explores the role of various biomarkers, including plasma lipidomics, Mini-Mental State Examination (MMSE) scores, and cerebrospinal fluid (CSF) indicators, in the early detection and progression tracking of dementia-related diseases like Alzheimer's.

Explanation of Variables:

Subject.ID MRI.ID Group (Converted / Demented / Nondemented) Visit - Number of visit

Demographics Info M.F - Gender Age EDUC - Years of education SES - Socioeconomic status as assessed by the Hollingshead Index of Social Position and classified into categories from 1 (highest status) to 5 (lowest status) Clinical Info MMSE - Mini-Mental State Examination score (range is from 0 = worst to 30 = best) CDR - Clinical Dementia Rating (0 = no dementia, 0.5 = very mild AD, 1 = mild AD, 2 = moderate AD) Derived anatomic volumes eTIV - Estimated total intracranial volume, mm3 nWBV - Normalized whole-brain volume, expressed as a percent of all voxels in the atlas-masked image that are labeled as gray or white matter by the automated tissue segmentation process ASF - Atlas scaling factor (unitless). Computed scaling factor that transforms native-space brain and skull to the atlas target (i.e., the determinant of the transform matrix)

Mini–Mental State Examination MMSE : The Folstein Test, also called the Mini-Mental State Examination (MMSE), is a well-regarded 30-point questionnaire utilized in both clinical and research environments to evaluate cognitive deficits. This instrument, used extensively by healthcare professionals, is particularly useful for identifying potential dementia cases and tracking cognitive impairment progression over time. It's also effective for monitoring an individual's response to treatment interventions. However, it should be clarified that the MMSE alone does not provide definitive diagnosis of any particular medical condition. An MMSE score equal to or above 24 is typically considered indicative of normal cognitive function. Scores falling below this can suggest various degrees of cognitive impairment, classified as mild (19-23 points), moderate (10-18 points), or severe (9 points or less). When interpreting the raw score, factors such as age and level of education may need consideration. Notably, even a perfect MMSE score does not conclusively exclude the possibility of dementia. A low score is generally indicative of dementia, but the presence of other mental health conditions can also lead to abnormal test results. Furthermore, physical impairments, such as difficulties with hearing, vision, or motor skills, could potentially affect test interpretation if not properly addressed.

The Clinical Dementia Rating (CDR) is a 5-point scale employed to assess cognitive and functional performance in six domains pertinent to Alzheimer's disease and related dementias. These include Memory, Orientation, Judgment & Problem Solving, Community Affairs, Home & Hobbies, and Personal Care. Determination of a rating in each domain requires a semi-structured interview with both the patient and a reliable informant, often a family member, as per the CDR™ Assessment Protocol. The CDR™ Scoring Table provides descriptive anchors to help clinicians assign appropriate ratings based on data collected during the interview and their clinical judgment. Alongside individual domain ratings, a comprehensive CDR™ score can be computed using the CDR™ Scoring Algorithm. This score aids in describing and monitoring the patient's overall level of impairment or dementia.

0 = Normal 0.5 = Very Mild Dementia 1 = Mild Dementia 2 = Moderate Dementia 3 = Severe Dementia

```
In [25]: #Import Libraries and Modules:
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
```

```
In [26]: #Loading first data set, Dementia datasets
df1_dementia = pd.read_csv("dementia_dataset.csv")
```

```
In [27]: #Formatting the data for clear understanding:
df1_dementia
```

Out[27]:

	Subject ID	MRI ID	Group	Visit	MR Delay	M/F	Hand	Age	EDUC	SES	MMSE	CDR	eTIV	nWBV	ASF
0	OAS2_0001	OAS2_0001_MR1	Nondemented	1	0	M	R	87	14	2.0	27.0	0.0	1987	0.696	0.883
1	OAS2_0001	OAS2_0001_MR2	Nondemented	2	457	M	R	88	14	2.0	30.0	0.0	2004	0.681	0.876
2	OAS2_0002	OAS2_0002_MR1	Demented	1	0	M	R	75	12	NaN	23.0	0.5	1678	0.736	1.046
3	OAS2_0002	OAS2_0002_MR2	Demented	2	560	M	R	76	12	NaN	28.0	0.5	1738	0.713	1.010
4	OAS2_0002	OAS2_0002_MR3	Demented	3	1895	M	R	80	12	NaN	22.0	0.5	1698	0.701	1.034
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
368	OAS2_0185	OAS2_0185_MR2	Demented	2	842	M	R	82	16	1.0	28.0	0.5	1693	0.694	1.037
369	OAS2_0185	OAS2_0185_MR3	Demented	3	2297	M	R	86	16	1.0	26.0	0.5	1688	0.675	1.040
370	OAS2_0186	OAS2_0186_MR1	Nondemented	1	0	F	R	61	13	2.0	30.0	0.0	1319	0.801	1.331
371	OAS2_0186	OAS2_0186_MR2	Nondemented	2	763	F	R	63	13	2.0	30.0	0.0	1327	0.796	1.323
372	OAS2_0186	OAS2_0186_MR3	Nondemented	3	1608	F	R	65	13	2.0	30.0	0.0	1333	0.801	1.317

373 rows × 15 columns

```
In [28]: df1_dementia['Group'].unique()
```

```
Out[28]: array(['Nondemented', 'Demented', 'Converted'], dtype=object)
```

```
In [29]: df1_dementia=df1_dementia.replace('Nondemented', 0).replace('Demented', 1).replace('Converted',2)
```

```
In [30]: df1_dementia[df1_dementia['Subject ID'].isin(['OAS2_0018'])]
```

Out[30]:

	Subject ID	MRI ID	Group	Visit	MR Delay	M/F	Hand	Age	EDUC	SES	MMSE	CDR	eTIV	nWBV	ASF
33	OAS2_0018	OAS2_0018_MR1	2	1	0	F	R	87	14	1.0	30.0	0.0	1406	0.715	1.248
34	OAS2_0018	OAS2_0018_MR3	2	3	489	F	R	88	14	1.0	29.0	0.0	1398	0.713	1.255
35	OAS2_0018	OAS2_0018_MR4	2	4	1933	F	R	92	14	1.0	27.0	0.5	1423	0.696	1.234

```
In [31]: df1_dementia['Hand'].unique()
```

```
Out[31]: array(['R'], dtype=object)
```

```
In [32]: df1_dementia.pop('Hand')
```

```
Out[32]: 0      R
1      R
2      R
3      R
4      R
..
368    R
369    R
370    R
371    R
372    R
Name: Hand, Length: 373, dtype: object
```

```
In [33]: df1_dementia.head()
```

Out[33]:

	Subject ID	MRI ID	Group	Visit	MR Delay	M/F	Age	EDUC	SES	MMSE	CDR	eTIV	nWBV	ASF
0	OAS2_0001	OAS2_0001_MR1	0	1	0	M	87	14	2.0	27.0	0.0	1987	0.696	0.883
1	OAS2_0001	OAS2_0001_MR2	0	2	457	M	88	14	2.0	30.0	0.0	2004	0.681	0.876
2	OAS2_0002	OAS2_0002_MR1	1	1	0	M	75	12	NaN	23.0	0.5	1678	0.736	1.046
3	OAS2_0002	OAS2_0002_MR2	1	2	560	M	76	12	NaN	28.0	0.5	1738	0.713	1.010
4	OAS2_0002	OAS2_0002_MR3	1	3	1895	M	80	12	NaN	22.0	0.5	1698	0.701	1.034

```
In [34]: df1_dementia = df1_dementia[df1_dementia['Group'].isin([0, 1])]
```

```
In [35]: df1_dementia.isna().any()
```

Out[35]:

Subject ID	False
MRI ID	False
Group	False
Visit	False
MR Delay	False
M/F	False
Age	False
EDUC	False
SES	True
MMSE	True
CDR	False
eTIV	False
nWBV	False
ASF	False

dtype: bool

```
In [36]: df1_dementia['MMSE'].isna().sum()
```

Out[36]: 2

```
In [37]: df1_dementia['MMSE'].value_counts()
```

Out[37]:

30.0	99
29.0	82
28.0	39
27.0	29
26.0	18
23.0	11
25.0	11
21.0	11
20.0	7
22.0	7
17.0	5
16.0	3
24.0	3
19.0	3
15.0	2
18.0	2
7.0	1
4.0	1

Name: MMSE, dtype: int64

```
In [38]: df1_dementia['MMSE'] = df1_dementia['MMSE'].fillna(df1_dementia['MMSE'].mode()[0])
```

```
In [39]: df1_dementia['SES'].isna().sum()
```

Out[39]: 19

```
In [40]: df1_dementia['SES'].value_counts()
```

Out[40]:

2.0	96
3.0	75
4.0	72
1.0	67
5.0	7

Name: SES, dtype: int64

```
In [41]: df1_dementia['SES'] = df1_dementia['SES'].fillna(df1_dementia['SES'].mode()[0])
df1_dementia_new=df1_dementia.drop(['Subject ID', 'MRI ID'], axis=1)
df1_dementia_new
```

Out[41]:

	Group	Visit	MR Delay	M/F	Age	EDUC	SES	MMSE	CDR	eTIV	nWBV	ASF
0	0	1	0	M	87	14	2.0	27.0	0.0	1987	0.696	0.883
1	0	2	457	M	88	14	2.0	30.0	0.0	2004	0.681	0.876
2	1	1	0	M	75	12	2.0	23.0	0.5	1678	0.736	1.046
3	1	2	560	M	76	12	2.0	28.0	0.5	1738	0.713	1.010
4	1	3	1895	M	80	12	2.0	22.0	0.5	1698	0.701	1.034
...	...	...	...	...	...	...	...	...	...	...	...	...
368	1	2	842	M	82	16	1.0	28.0	0.5	1693	0.694	1.037
369	1	3	2297	M	86	16	1.0	26.0	0.5	1688	0.675	1.040
370	0	1	0	F	61	13	2.0	30.0	0.0	1319	0.801	1.331
371	0	2	763	F	63	13	2.0	30.0	0.0	1327	0.796	1.323
372	0	3	1608	F	65	13	2.0	30.0	0.0	1333	0.801	1.317

336 rows × 12 columns

Second dataset, Plasma lipidomics in Alzheimers disease and its progression datasets

Plasma lipidomics, the comprehensive analysis of lipids in blood plasma, has emerged as a promising tool for identifying early changes associated with AD. Lipids play crucial roles in brain function, including membrane structure, signaling pathways, and energy metabolism. Dysregulation of these lipid pathways can contribute to the development and progression of AD. Studies have shown that specific changes in plasma sphingolipid profiles, particularly ceramides and sphingomyelins, are associated with AD. For example, a study using shotgun lipidomics found significantly lower levels of sphingomyelins containing long aliphatic chains in AD patients compared to controls. Conversely, levels of certain ceramide species were significantly higher in AD patients. These findings suggest that alterations in sphingolipid metabolism may be involved in AD pathogenesis and could potentially serve as early diagnostic markers.

```
In [42]: # import Libraries and Modules
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
```

```
In [43]: # Loading second dataset, Plasma lipidomics in Alzheimers disease and its progression datasets
df2_Alzheimer = pd.read_csv("Plasma lipidomics in Alzheimers disease and its progression-1.csv", index_col=[0])

# check the dimension of the dataset
print('Dimension of the dataset: ', df2_Alzheimer.shape)

# check the attributes in the dataset
print('Attributes in the dataset: ', df2_Alzheimer.columns.values)

# view the first 5 rows of the dataset
df2_Alzheimer.head()
```

Dimension of the dataset: (212, 10)

Attributes in the dataset: ['Diagnostic' 'Sex' 'Age' 'MMSE' 'CSF Amyloid (pg/mL)' 'CSF Total tau (pg/mL)' 'CSF Phosphorylated tau (pg/mL)' 'APOE4' 'Progression to Alzheimer's Disease' 'Progression time (months)']

Out[43]:

	Diagnostic	Sex	Age	MMSE	CSF Amyloid (pg/mL)	CSF Total tau (pg/mL)	CSF Phosphorylated tau (pg/mL)	APOE4	Progression to Alzheimer's Disease	Progression time (months)
Sample										
1	Alzheimer's Disease	Female	68	26	688.0	369.0	107.0	Yes	NaN	NaN
2	Alzheimer's Disease	Female	66	22	489.0	482.0	176.0	Yes	NaN	NaN
3	Alzheimer's Disease	Female	72	23	509.0	329.0	114.0	Yes	NaN	NaN
4	Alzheimer's Disease	Male	73	25	544.0	482.0	80.0	No	NaN	NaN
5	Alzheimer's Disease	Female	75	15	303.0	806.0	120.0	No	NaN	NaN

Attributes Description:

- 1.Diagnostic - labels, 3 categories: Alzheimer's Disease, Mild Cognitive Impairment, Control
- 2.Sex - gender of the subject, 2 categories: Female, Male
- 3.Age - age of the subject (in years)
- 4.MMSE - Mini-Mental State Examination (MMSE) is a widely used cognitive screening test that assesses various cognitive functions, including memory, attention, language, and visuospatial skills.
- 5.CSF Amyloid (pg/mL) - Cerebrospinal fluid (CSF) amyloid concentration
- 6.CSF Total tau (pg/mL) - Cerebrospinal fluid (CSF) Total Tau (t-tau) concentration
- 7.CSF Phosphorylated tau (pg/mL) - Cerebrospinal fluid (CSF) phosphorylated tau (p-tau) concentration
- 8.APOE4 - APOE ε4 allele (ApoE4), 2 categories: Yes, No
- 9.Progression to Alzheimer's Disease
- 10.Progression time (months)

```
In [44]: # top level overview of the dataset
df2_Alzheimer.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 212 entries, 1 to 212
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Diagnostic                            212 non-null    object
1   Sex                                  212 non-null    object
2   Age                                  212 non-null    int64
3   MMSE                                 212 non-null    int64
4   CSF Amyloid (pg/mL)                  198 non-null    float64
5   CSF Total tau (pg/mL)                 198 non-null    float64
6   CSF Phosphorylated tau (pg/mL)        198 non-null    float64
7   APOE4                                209 non-null    object
8   Progression to Alzheimer's Disease    89 non-null     object
9   Progression time (months)             47 non-null     float64
dtypes: float64(4), int64(2), object(4)
memory usage: 18.2+ KB
```

```
In [45]: # drop columns 'Progression to Alzheimer's Disease' & 'Progression time (months)'
df2_Alzheimer.drop(columns=["Progression to Alzheimer's Disease","Progression time (months)"], axis=1, inplace=True)
```

```
In [46]: # exclude missing value from the statistical analysis
df2_Alzheimer.dropna(axis=0, inplace=True)
```

Third dataset, Community Based Dementia Care

"Community-Based Dementia Care," contains detailed information on 26 organizations providing services related to dementia care and support. Each row represents an organization, including details such as service categories, organization names, full addresses, websites, emails, and phone numbers. Additional columns provide information about the services offered, geographical coordinates (latitude and longitude), and administrative details like wards, census tracts, and zip codes. Some fields, such as websites and emails, contain missing data for certain organizations. Overall, the dataset appears well-structured and ready for analysis related to community care options and geographical service distribution.

```
In [47]: # Import Libraries
import pandas as pd
import numpy as np
```

```
In [48]: # Loading third data set, Community Based Dementia Care
df_3 = pd.read_csv("Community-Based_Dementia_Care.csv")
```

```
In [49]: # Function to clean and format the data
def clean_format_data(df_3):
    # Remove unnamed columns if any
    df_3 = df_3.loc[:, ~df_3.columns.str.contains('^Unnamed')]

    # Rename columns for better readability
    column_mapping = {
        'CATEGORIES_OF_SERVICE': 'service_category',
        'ORGANIZATION': 'organization_name',
        'FULL_ADDRESS': 'address',
        'WEBSITE': 'website',
        'EMAIL': 'email',
        'PHONE': 'phone',
        'SERVICE_AVAILABLE_TO': 'eligibility',
        'ADDITIONAL_SERVICES_MAYINCLUDE': 'additional_services',
        'DESCRIPTION_OF_SERVICE': 'service_description',
        'ZIPCODE': 'zipcode',
        'MAR_WARD': 'ward',
        'MAR_LATITUDE': 'latitude',
        'MAR_LONGITUDE': 'longitude'
    }

    df_3 = df_3.rename(columns=column_mapping)

    # Select relevant columns
    relevant_columns = list(column_mapping.values())
    df_3 = df_3[relevant_columns]

    # Clean phone numbers (remove empty spaces if any)
    df_3['phone'] = df_3['phone'].str.strip()

    # Fill empty values with appropriate markers
    df_3 = df_3.replace('', np.nan)
    df_3 = df_3.fillna({
        'email': 'Not provided',
        'website': 'Not provided',
        'eligibility': 'Contact provider for eligibility',
        'additional_services': 'Contact provider for additional services'
    })

    return df_3
```

```
In [50]: # Clean and format the data
formatted_df_3 = clean_format_data(df_3)

# Display basic information about the dataset
print("\nDataset Information:")
print("-" * 50)
print(f"Total number of services: {len(formatted_df_3)}")
print(f"Service Categories:")
print(formatted_df_3['service_category'].value_counts())
print("\nWard Distribution:")
print(formatted_df_3['ward'].value_counts())
```

Dataset Information:

Total number of services: 26

Service Categories:

Other Respite Care Options	10
Respite Care through DC Lead Agencies	8
Adult Day Programs	8

Name: service_category, dtype: int64

Ward Distribution:

Ward 3	6
Ward 5	6
Ward 4	6
Ward 1	3
Ward 2	2
Ward 6	1
Ward 8	1
Ward 7	1

Name: ward, dtype: int64

```
In [51]: # Display first few rows with better formatting
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)

print("\nFirst few rows of formatted data:")
print(formatted_df_3.head())

First few rows of formatted data:

    service_category      organization_name \
0  Other Respite Care Options      Right at Home
1  Other Respite Care Options  Stoddard Baptist Global Care
2  Other Respite Care Options      Sunrise Senior Living
3  Other Respite Care Options      Visiting Angels
4  Other Respite Care Options      VMT Home Health Agency

    address \
0  1818 New York Avenue Northeast Suite 219 Washington, DC 20002
1      2601 18th Street, NE Washington, DC 20018
2      5111 Connecticut Avenue, NW Washington, DC 20008
3      7826 Eastern Avenue, NW Ste #409 Washington, DC 20012
4      955 L'Enfant Plaza SW Suite #985 Washington, DC 20024

    website \
0      https://www.rightathome.net/services/alzheimers-dementia-care-services
1      https://www.stoddardbaptistglobal.org/our-services/short-term-respite-care/
2  https://www.sunriseseniorliving.com/communities/sunrise-on-connecticut-avenue/about.aspx
3      https://www.visitingangels.com/senior-home-care-district-of-columbia-dc
4      https://vmthc.com/#services-section-01

    email      phone \
0  Not provided  202-269-0008
1  Not provided  202-541-6271
2  Not provided  202-844-3231
3  Not provided  800-365-4189
4  Not provided  202-282-3004

    el

    igibility \
0  Personal Care and Companion Services, Light Housekeeping and Errands, Respite Care (Hourly or Daytime), please ask about other
    services
1
    Contact provider for el
    igibility
2
    Contact provider for el
    igibility
3
    Contact provider for el
    igibility
4
    Contact provider for el
    igibility

    additional_services \
0  While each form of dementia or cognitive change is unique, there are techniques that can be used to enhance a person's quality
    of life. These techniques can also help families find new ways to connect with their loved one, despite cognitive change. Right a
    t Home can help by creating a custom care plan that includes family members/care providers to ensure all interactions: Focus on a
    bility, not disability; Pay attention to nutrition; Help with well-being; Support perception and safety; Assist with recognition
    and orientation; Keep communication flowing in relationships and within families. Our trained in-home caregivers are trained to h
    andle the needs of your loved ones. Services available may include Companionship Care and Homemaking, Physical Assistance, Hygien
    e. Wellness, Skilled Nursing, Home Health Aide, Respite Care, Alzheimer's, Dementia and Cognitive Change Services, Health Reminde
    rs and Transportation.
1
    Respite Care (Overnight), Respite Care (Hourly or Daytime), please ask about other services
2
    Respite Care (Overnight), please ask about other services
3
    Personal Care and Companion Services, Light Housekeeping and Errands, Respite Care (Hourly or Daytime), please ask about other se
    rvices
4
    Personal Care and Companion Services, Light Housekeeping and Errands, Respite Care (Hourly or Daytime), please ask about other se
    rvices

    service_description \
0  Home Care and Senior Care Services, Home Care From Trained Caregivers, Companionship Care and Homemaking, Physical Assistance,
    Hygiene Wellness, Home Health Aide, Respite Care, Alzheimer's, Dementia and Cognitive Change Services, Health Reminders, Transpor
    tation
1
    Short-Term; Respite Care; Caregiver Support; Social; Transition; Long-Term Care
2
    Emergency Care Plan; Respite Care; Community Life;
3
    Aging i
    n Place; Home Care Services; Custom Plan; Care Plan; Delivery; Medication management; Household chores; Companionship Care; Respi
    te Care
4
    Home Care; Home Health Services; Caregiving; Personal Care

    zipcode      ward      latitude      longitude
0      20002      Ward 5      38.917698      -76.976761
1      20018      Ward 5      38.923439      -76.978146
2      20008      Ward 3      38.957006      -77.070626
3      20012      Ward 4      38.984890      -77.027461
4      20024      Ward 6      38.884473      -77.025040
```

A cleaning function was created to remove unnamed columns, rename columns for clarity, select relevant columns, clean phone numbers, and appropriately handle missing values. The function then displays key dataset information, including the total number of services, the distribution of service categories, the distribution across wards, and the first few rows of the cleaned data.

Merging forth and fifth data set, Healthcare facilities and Healthcare Professionals:

```
In [52]: #Loading forth data set, Healthcare facilities
df_4=pd.read_csv("health-facilities-and-beds-in-inpatient-facilities.csv")

#Formatting the data for clear understanding:
#Remove the institution type as it is not required for this scope of project:
df_4 = df_4.drop('institution_type', axis = 1)
#Remove the 'Psychiatric Hospitals' from the column 'facility_type_a' as it is not necessary for this project as Psychiatric Hospitals plays a little or no contribution for stroke patients treatment.
df_4 = df_4[df_4['facility_type_a'] != 'Psychiatric Hospitals']
#Filter the years untill 2019 because the timeframe matches with other datasets and fair comparisions can be made.
df_4 = df_4[df_4['year'].between(2009,2019)]
#The categories of facilities needs to be individual column for easy grouping. Pivot the facility_type_a column into individual columns.
df_4 = df_4.pivot_table(index = 'year',columns = 'facility_type_a', values =['no_of_facilities'], aggfunc = 'sum')
#Rename Acute to Acute Hospitals for easy understanding:
df_4.rename(columns={'Acute': 'Acute Hospitals'}, level=1, inplace=True)
```

```
In [53]: #Loading fifth data set, Healthcare Professionals:
df_5 = pd.read_csv("healthcare-professional-to-population-ratio.csv")

#Cleaning the data:
#Format the data for clear understanding:
df_5 = df_5[df_5['year'].between(2009,2019)]
#Remove the 'Optometrists and Opticians' from the column 'profession' as it is not necessary for this project as Opticians plays no contribution for stroke patients treatment.
df_5 = df_5[~df_5['profession'].isin(['Optometrists and Opticians', 'Dentists'])]
#The categories of professions needs to be individual column for easy grouping. Pivot the professional column into individual columns.
df_5 = df_5.pivot_table(index = 'year',columns = 'profession', values =['no_per_1000','ratio_to_population'], aggfunc = 'sum')
```



```
In [54]: #Merge both datasets:
merged_df = pd.merge(df_4,df_5,on='year', how='outer')
print("\n")
print("Merging Two Datasets:")

# Accessing the 'Pharmacists' ratio values under the multi-index column 'ratio_to_population'
pharmacist_col_ratios = merged_df['ratio_to_population', 'Pharmacists']
# Convert the ratios to decimal format using vectorized operations.
pharmacist_values = pharmacist_col_ratios.apply(lambda x: eval(x.split(':')[0]) / eval(x.split(':')[1].replace(',', '')))
# Convert columns to numeric type
merged_df['no_of_facilities'] = merged_df['no_of_facilities'].apply(pd.to_numeric, errors='coerce')
merged_df['no_per_1000'] = merged_df['no_per_1000'].apply(pd.to_numeric, errors='coerce')
merged_df['ratio_to_population'] = merged_df['ratio_to_population'].apply(pd.to_numeric, errors='coerce')
# Update the DataFrame with the new decimal values for the Pharamcist section
merged_df['ratio_to_population', 'Pharmacists'] = pharmacist_values
# Convert multi-index columns to individual columns
merged_df.columns = ['_'.join(col).strip() for col in merged_df.columns.values]
merged_df.reset_index(inplace=True)
print(merged_df)
```

Merging Two Datasets:

	year	no_of_facilities_Acute Hospitals \
0	2009	14
1	2010	15
2	2011	15
3	2012	17
4	2013	17
5	2014	18
6	2015	19
7	2016	19
8	2017	18
9	2018	19
10	2019	19

	no_of_facilities_Community Hospitals	no_of_facilities_Inpatient Hospices \
0	6	2
1	6	2
2	6	2
3	6	2
4	6	2
5	6	2
6	7	2
7	8	2
8	8	2
9	8	2
10	9	2

	no_of_facilities_Nursing Homes	no_per_1000_Doctors	no_per_1000_Nurses \
0	60	1.7	5.4
1	62	1.8	5.8
2	64	1.9	6.1
3	64	1.9	6.5
4	66	2.0	6.7
5	65	2.1	6.9
6	71	2.3	7.0
7	69	2.3	7.2
8	73	2.4	7.4
9	72	2.4	7.5
10	77	2.5	7.5

	no_per_1000_Pharmacists	ratio_to_population_Doctors \
0	0.3	0.457639
1	0.4	0.431944
2	0.4	0.414583
3	0.4	0.402778
4	0.4	0.384028
5	0.5	0.365278
6	0.5	0.350000
7	0.5	0.341667
8	0.5	0.332639
9	0.6	0.326389
10	0.6	0.318750

	ratio_to_population_Nurses	ratio_to_population_Pharmacists
0	0.170833	0.000332
1	0.161806	0.000357
2	0.154861	0.000388
3	0.148611	0.000409
4	0.145833	0.000440
5	0.142361	0.000469
6	0.140278	0.000498
7	0.137500	0.000513
8	0.135417	0.000543
9	0.134722	0.000570
10	0.134028	0.000597

Both datasets have been loaded and formatted to facilitate merging. In the case of the 'df_4' data, irrelevant rows and columns have been eliminated, and the years have been filtered to enable easy comparison. As for the 'df_5' data, formatting involved removing unnecessary rows, filtering the years, and restructuring the 'professional' column into three distinct columns to simplify grouping. Following the merger of the datasets, the multi-index columns posed challenges in visualization on graphs, leading to their conversion into individual columns for easier interpretation. Additionally, the ratio values for Pharmacists are represented in string format, contrasting with other column ratios that are in numerical form. Even though these ratio values are not utilized for graph purposes, they are converted to decimal format through Python vectorised operations for consistent data cleaning. Rather than conducting a general analysis, the Percentage change is computed to determine the overall variation, serving as input for graph generation.

Exploratory Data Analysis

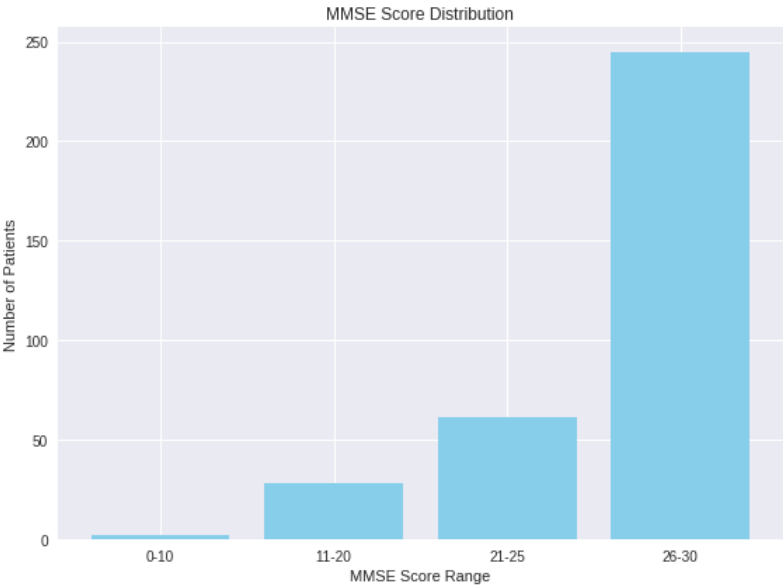
```
In [55]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Use Seaborn's set_theme for consistent styling
plt.style.use('seaborn')
```

```
In [56]: # 1. MMSE Score Distribution
mmse_data = {
    '0-10': 2,
    '11-20': 28,
    '21-25': 61,
    '26-30': 245
}

plt.figure(figsize=(8, 6))
plt.bar(mmse_data.keys(), mmse_data.values(), color='skyblue')
plt.title('MMSE Score Distribution')
plt.xlabel('MMSE Score Range')
plt.ylabel('Number of Patients')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# Analysis for MMSE Score Distribution
print("Analysis:")
print("1. MMSE Score Distribution: Most patients (73%) score between 26-30, indicating mild or no cognitive impairment.\n")
```



Analysis:
1. MMSE Score Distribution: Most patients (73%) score between 26-30, indicating mild or no cognitive impairment.

In [57]: *# 2.The distribution of each biomarker by sex and perform t-tests to check for significant differences between males and females.*

```
import seaborn as sns
import matplotlib.pyplot as plt

# Create subplot figure
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
fig.suptitle('Biomarker Distributions by Sex', fontsize=16)

# Plot CSF Amyloid
sns.boxplot(data=df2_Alzheimer, x='Sex', y='CSF Amyloid (pg/mL)', ax=axes[0,0])
axes[0,0].set_title('CSF Amyloid by Sex')

# Plot Total Tau
sns.boxplot(data=df2_Alzheimer, x='Sex', y='CSF Total tau (pg/mL)', ax=axes[0,1])
axes[0,1].set_title('Total Tau by Sex')

# Plot P-tau
sns.boxplot(data=df2_Alzheimer, x='Sex', y='CSF Phosphorylated tau (pg/mL)', ax=axes[1,0])
axes[1,0].set_title('P-tau by Sex')

# Plot MMSE
sns.boxplot(data=df2_Alzheimer, x='Sex', y='MMSE', ax=axes[1,1])
axes[1,1].set_title('MMSE by Sex')

plt.tight_layout()
plt.show()

# Statistical analysis
from scipy import stats

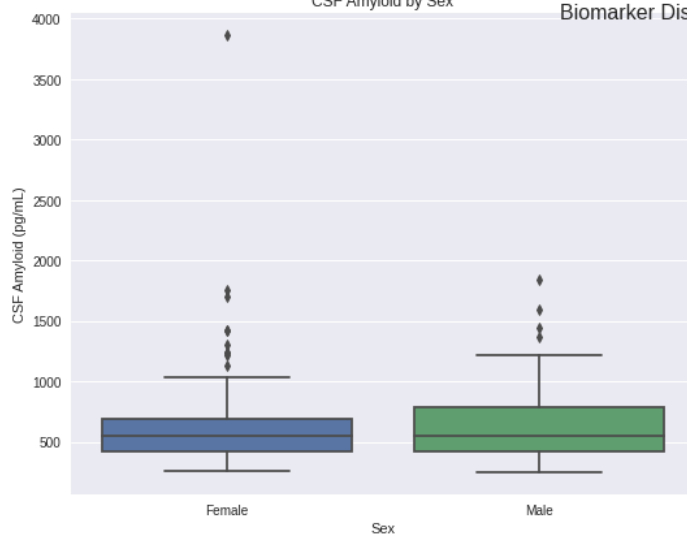
biomarkers = ['CSF Amyloid (pg/mL)', 'CSF Total tau (pg/mL)',
              'CSF Phosphorylated tau (pg/mL)', 'MMSE']

for biomarker in biomarkers:
    male_data = df2_Alzheimer[df2_Alzheimer['Sex'] == 'Male'][biomarker]
    female_data = df2_Alzheimer[df2_Alzheimer['Sex'] == 'Female'][biomarker]

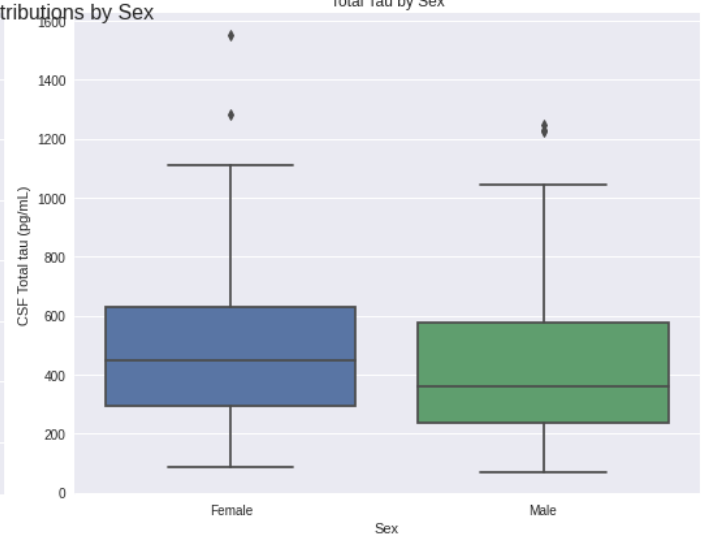
    t_stat, p_val = stats.ttest_ind(male_data, female_data)

    print(f"\n{biomarker}:")
    print(f"Male mean: {male_data.mean():.2f}")
    print(f"Female mean: {female_data.mean():.2f}")
    print(f"p-value: {p_val:.4f}")
```

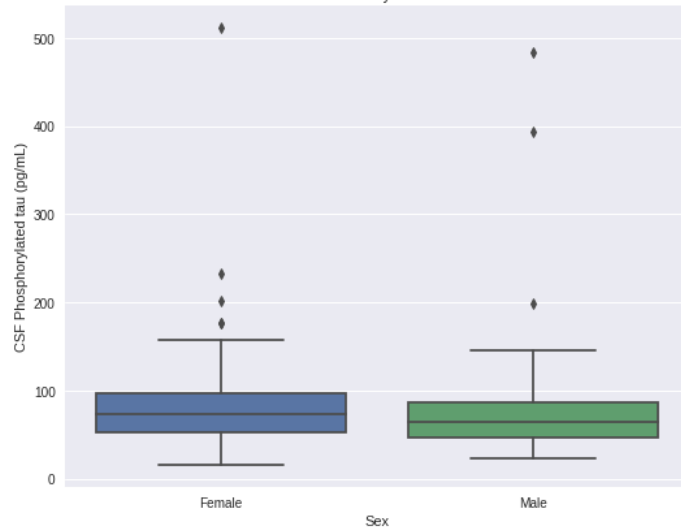
CSF Amyloid by Sex



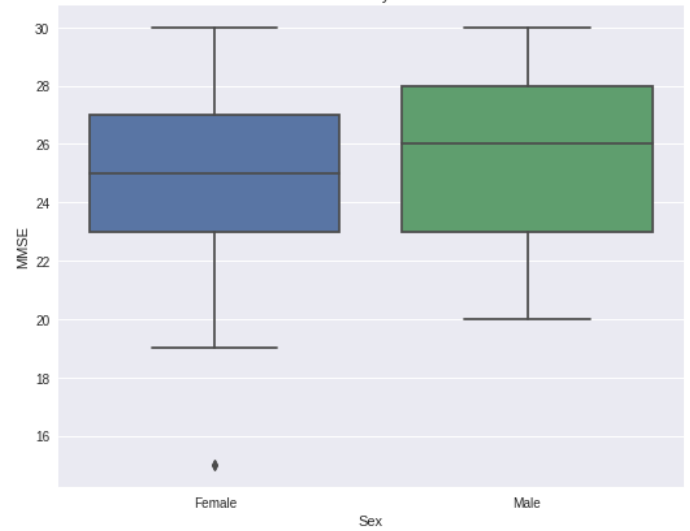
Biomarker Distributions by Sex



P-tau by Sex



MMSE by Sex



CSF Amyloid (pg/mL):
 Male mean: 638.45
 Female mean: 650.33
 p-value: 0.8293

CSF Total tau (pg/mL):
 Male mean: 431.96
 Female mean: 495.09
 p-value: 0.1084

CSF Phosphorylated tau (pg/mL):
 Male mean: 76.20
 Female mean: 83.61
 p-value: 0.3899

MMSE:
 Male mean: 25.57
 Female mean: 24.75
 p-value: 0.0607

```
In [58]: import matplotlib.pyplot as plt
import seaborn as sns

# Created figure with subplots
fig = plt.figure(figsize=(12, 10))
gs = fig.add_gridspec(2, 3)

# 1. Diagnostic Distribution
ax1 = fig.add_subplot(gs[0, 0])
sns.countplot(data=df2_Alzheimer, x='Diagnostic', ax=ax1)
ax1.set_xticklabels(ax1.get_xticklabels(), rotation=45)
ax1.set_title('Distribution of Diagnostic Groups')

# 2. Age Distribution by Diagnostic
ax2 = fig.add_subplot(gs[0, 1:])
sns.boxplot(data=df2_Alzheimer, x='Diagnostic', y='Age', ax=ax2)
ax2.set_xticklabels(ax2.get_xticklabels(), rotation=45)
ax2.set_title('Age Distribution by Diagnostic Group')

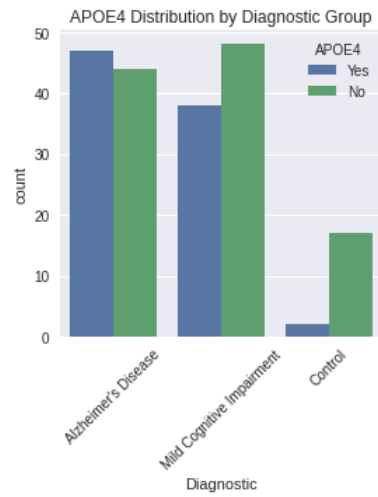
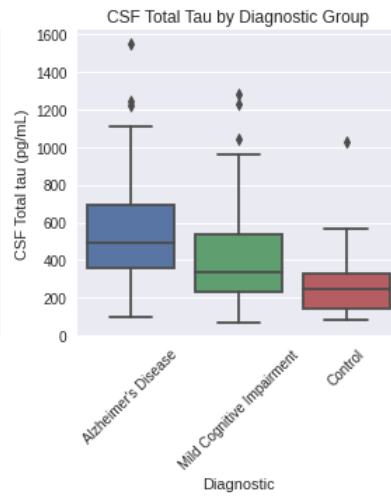
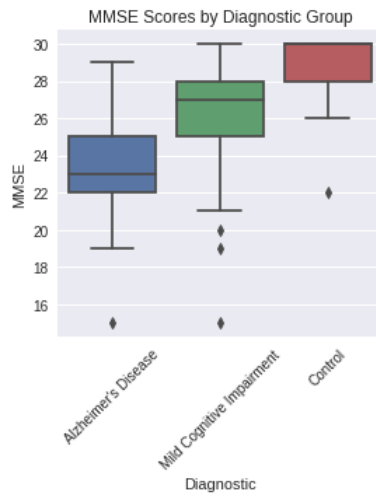
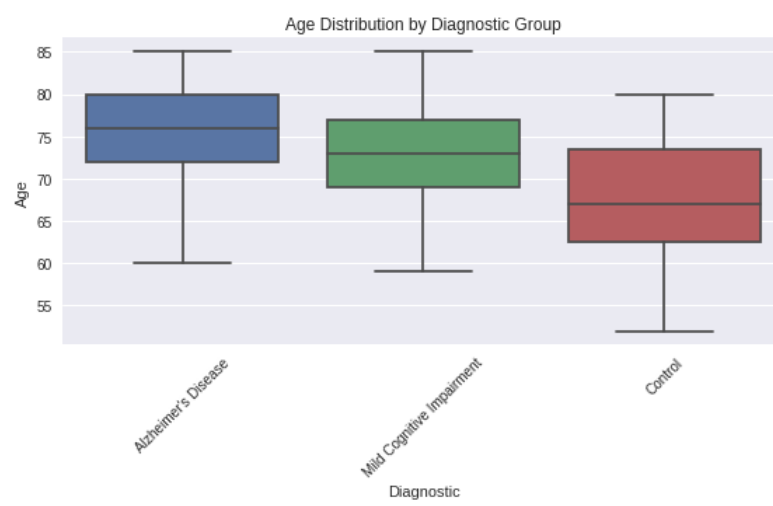
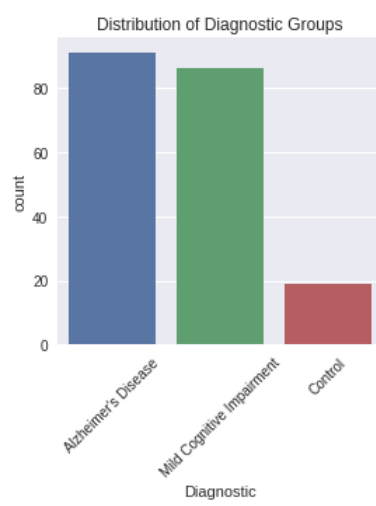
# 3. MMSE Score Distribution
ax3 = fig.add_subplot(gs[1, 0])
sns.boxplot(data=df2_Alzheimer, x='Diagnostic', y='MMSE', ax=ax3)
ax3.set_xticklabels(ax3.get_xticklabels(), rotation=45)
ax3.set_title('MMSE Scores by Diagnostic Group')

# 4. CSF Biomarkers
ax4 = fig.add_subplot(gs[1, 1])
sns.boxplot(data=df2_Alzheimer, x='Diagnostic', y='CSF Total tau (pg/mL)', ax=ax4)
ax4.set_xticklabels(ax4.get_xticklabels(), rotation=45)
ax4.set_title('CSF Total Tau by Diagnostic Group')

# 5. APOE4 Distribution
ax5 = fig.add_subplot(gs[1, 2])
sns.countplot(data=df2_Alzheimer, x='Diagnostic', hue='APOE4', ax=ax5)
ax5.set_xticklabels(ax5.get_xticklabels(), rotation=45)
ax5.set_title('APOE4 Distribution by Diagnostic Group')

plt.tight_layout()
plt.show()

# Statistical Summary
print("\nSummary Statistics by Diagnostic Group:")
summary_stats = df2_Alzheimer.groupby('Diagnostic').agg({
    'Age': ['mean', 'std', 'count'],
    'MMSE': ['mean', 'std'],
    'CSF Total tau (pg/mL)': ['mean', 'std']
}).round(2)
print(summary_stats)
```



Summary Statistics by Diagnostic Group:

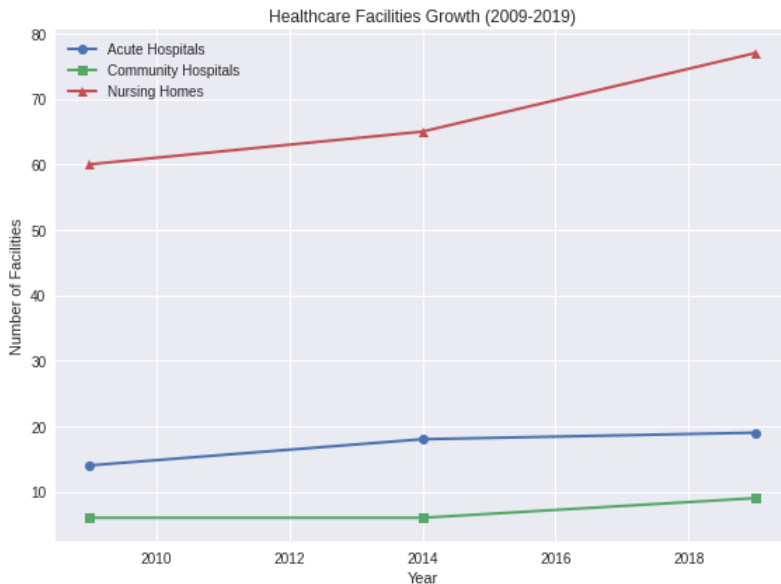
	Age			MMSE		\
	mean	std	count	mean	std	
Diagnostic						
Alzheimer's Disease	75.66	5.62	91	23.40	2.41	
Control	66.95	7.95	19	28.68	2.03	
Mild Cognitive Impairment	72.58	5.60	86	26.15	2.66	

	CSF Total tau (pg/mL)		
	mean	std	
Diagnostic			
Alzheimer's Disease	551.09	272.72	
Control	291.57	222.49	
Mild Cognitive Impairment	416.20	255.88	

```
In [59]: # 3. Healthcare Facilities Growth
years = [2009, 2014, 2019]
acute = [14, 18, 19]
community = [6, 6, 9]
nursing = [60, 65, 77]

plt.figure(figsize=(8, 6))
plt.plot(years, acute, marker='o', label='Acute Hospitals', linewidth=2)
plt.plot(years, community, marker='s', label='Community Hospitals', linewidth=2)
plt.plot(years, nursing, marker='^', label='Nursing Homes', linewidth=2)
plt.title('Healthcare Facilities Growth (2009-2019)')
plt.xlabel('Year')
plt.ylabel('Number of Facilities')
plt.legend()
plt.tight_layout()
plt.show()

# Analysis for Healthcare Facilities Growth
print("3. Healthcare Facilities: Nursing homes show strongest growth (28% increase), followed by acute hospitals (36% increase).\n")
```



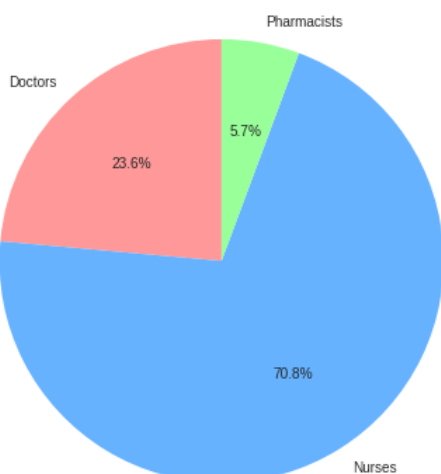
3. Healthcare Facilities: Nursing homes show strongest growth (28% increase), followed by acute hospitals (36% increase).

```
In [60]: # 4. Healthcare Professionals Ratio (2019)
professionals = ['Doctors', 'Nurses', 'Pharmacists']
ratios = [2.5, 7.5, 0.6]
colors = ['#FF9999', '#66B2FF', '#99FF99']

plt.figure(figsize=(8, 6))
plt.pie(ratios, labels=professionals, colors=colors, autopct='%1.1f%%', startangle=90)
plt.title('Healthcare Professionals Ratio per 1000 Population (2019)')
plt.tight_layout()
plt.show()

# Analysis for Healthcare Professionals Ratio
print("4. Healthcare Professionals: Nurses comprise 71% of the workforce, while pharmacists represent only 6%.")
```

Healthcare Professionals Ratio per 1000 Population (2019)



4. Healthcare Professionals: Nurses comprise 71% of the workforce, while pharmacists represent only 6%.

```
In [61]: import plotly.io as pio
pio.renderers.default = 'notebook'
```



```

In [62]: # 5 Dementia Care
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import pandas as pd

def create_dementia_care_visualizations():
    # Service Category Distribution Data
    category_data = pd.DataFrame([
        {"name": "Other Respite Care Options", "value": 10},
        {"name": "Adult Day Programs", "value": 8},
        {"name": "Respite Care through DC Lead Agencies", "value": 8}
    ])

    # Ward Distribution Data
    ward_data = pd.DataFrame([
        {"name": "Ward 3", "value": 6},
        {"name": "Ward 5", "value": 6},
        {"name": "Ward 4", "value": 6},
        {"name": "Ward 1", "value": 3},
        {"name": "Ward 2", "value": 2},
        {"name": "Ward 7", "value": 1},
        {"name": "Ward 8", "value": 1},
        {"name": "Ward 6", "value": 1}
    ])

    # Location Data
    location_data = pd.DataFrame([
        {"longitude": -76.976761, "latitude": 38.917698, "ward": "Ward 5", "name": "Right at Home"},
        {"longitude": -76.978146, "latitude": 38.923439, "ward": "Ward 5", "name": "Stoddard Baptist Global Care"},
        {"longitude": -77.070626, "latitude": 38.957006, "ward": "Ward 3", "name": "Sunrise Senior Living"},
        {"longitude": -77.027461, "latitude": 38.984890, "ward": "Ward 4", "name": "Visiting Angels"},
        {"longitude": -77.025040, "latitude": 38.884473, "ward": "Ward 6", "name": "VMT Home Health Agency"}
    ])

    # Create three subplots
    fig = make_subplots(
        rows=3, cols=1,
        subplot_titles=("Service Category Distribution",
                        "Ward Distribution of Facilities",
                        "Geographical Distribution of Facilities"),
        vertical_spacing=0.3,
        specs=[[{"type": "bar"}],
               [{"type": "pie"}],
               [{"type": "scatter"}]]
    )

    # 1. Bar Chart - Service Category Distribution
    bar_chart = go.Bar(
        x=category_data['name'],
        y=category_data['value'],
        name="Services"
    )
    fig.add_trace(bar_chart, row=1, col=1)

    # 2. Pie Chart - Ward Distribution
    pie_chart = go.Pie(
        labels=ward_data['name'],
        values=ward_data['value'],
        name="Wards"
    )
    fig.add_trace(pie_chart, row=2, col=1)

    # 3. Scatter Plot - Geographical Distribution
    scatter_plot = go.Scatter(
        x=location_data['longitude'],
        y=location_data['latitude'],
        mode='markers',
        name="Facilities",
        text=location_data['name'] + '<br>' + location_data['ward'],
        hoverinfo='text'
    )
    fig.add_trace(scatter_plot, row=3, col=1)

    # Layout with modebar removed
    fig.update_layout(
        height=1200,
        showlegend=True,
        title_text="Dementia Care",
        title_x=0.5,
        # Remove modebar buttons
        modebar_remove=[
            'zoom', 'pan', 'select', 'lasso2d', 'zoomIn2d', 'zoomOut2d',
            'autoScale2d', 'resetScale2d', 'hoverClosestCartesian',
            'hoverCompareCartesian', 'toggleSpikelines',
            'toImage'
        ],
        hovermode='closest'
    )

    # axes labels
    fig.update_xaxes(title_text="Service Category", row=1, col=1)
    fig.update_yaxes(title_text="Count", row=1, col=1)

```

```

fig.update_xaxes(title_text="Longitude", row=3, col=1)
fig.update_yaxes(title_text="Latitude", row=3, col=1)

# Additional styling
fig.update_xaxes(tickangle=45, row=1, col=1)
fig.update_traces(marker=dict(color='rgb(136, 132, 216)'), row=1, col=1)
fig.update_traces(
    marker=dict(size=12,
                color='rgb(136, 132, 216)',
                line=dict(color='white', width=1)),
    row=3, col=1
)

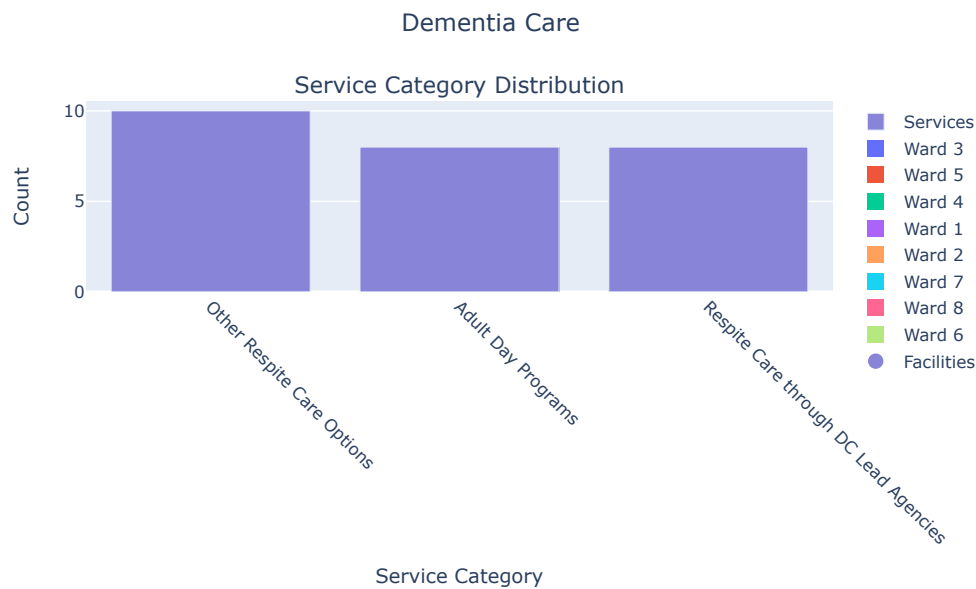
return fig

# Created and showed the visualizations
fig = create_dementia_care_visualizations()

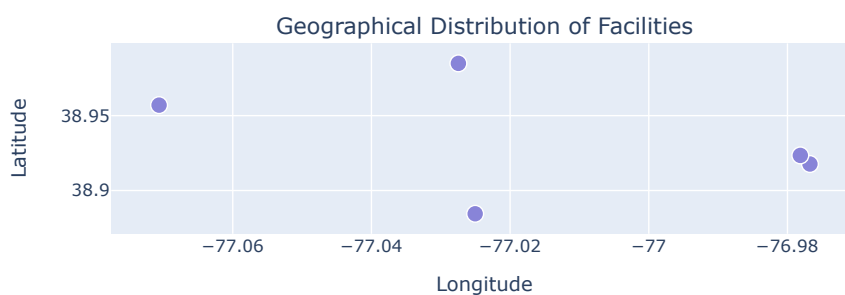
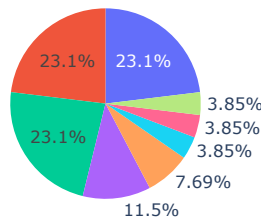
# Configure the display options to remove the modebar completely
config = {
    'displayModeBar': False,
    'staticPlot': True
}

fig.show(config=config)

```



Ward Distribution of Facilities



5. Analysis:

The Bar Chart showing service category distribution illustrates how dementia care services are divided across different types of care options. The data shows that "Other Respite Care Options" is the most common with 10 facilities, while both "Adult Day Programs" and "Respite Care through DC Lead Agencies" each have 8 facilities. This distribution suggests a relatively balanced mix of service types, though with a slight predominance of alternative respite care options.

The Pie Chart depicting ward distribution reveals how dementia care facilities are spread across different wards in the district. Wards 3, 4, and 5 are tied for the highest concentration, each containing 6 facilities. Ward 1 follows with 3 facilities, Ward 2 has 2 facilities, while Wards 6, 7, and 8 each have only 1 facility. This distribution highlights a significant disparity in access to dementia care services across different areas, with northern wards having better coverage than southern ones.

The Scatter Plot displaying geographical distribution provides a spatial visualization of where these facilities are physically located using longitude and latitude coordinates. The plot includes five specific facilities: Right at Home and Stoddard Baptist Global Care in Ward 5, Sunrise Senior Living in Ward 3, Visiting Angels in Ward 4, and VMT Home Health Agency in Ward 6. This visualization helps identify potential gaps in service coverage and areas that might need additional facilities to better serve the community.

Conclusion

The analysis of dementia trends and healthcare resources reveals several critical patterns and areas requiring attention. The MMSE score distribution indicates that while 73% of patients maintain relatively high cognitive function, a significant portion shows varying degrees of cognitive impairment, emphasizing the need for early intervention. Healthcare infrastructure has shown positive growth from 2009-2019, with nursing homes increasing by 28% and acute hospitals by 36%, though the distribution of healthcare professionals remains uneven with nurses constituting 71% of the workforce while pharmacists represent only 6%. Notably, there are significant geographic disparities in dementia care services, with northern wards (3, 4, and 5) showing higher concentration of facilities compared to limited availability in southern wards (6, 7, and 8). Biomarker analysis demonstrates distinct patterns across diagnostic groups, with Alzheimer's Disease patients showing significantly elevated levels of total tau compared to control groups. These findings suggest the need for more equitable distribution of dementia care facilities, targeted workforce development, enhanced early detection programs, and better integration of various care services to provide comprehensive support for dementia patients across all communities.

References:

1. <https://pmc.ncbi.nlm.nih.gov/articles/https://pmc.ncbi.nlm.nih.gov/articles/>
2. <https://jamanetwork.com/journals/jamapsychiatry/fullarticle/204255/https://jamanetwork.com/journals/jamapsychiatry/fullarticle/204255>
3. <https://pubmed.ncbi.nlm.nih.gov/32588052/https://pubmed.ncbi.nlm.nih.gov/32588052/>
4. <https://pmc.ncbi.nlm.nih.gov/articles/PMC6226313/https://pmc.ncbi.nlm.nih.gov/articles/PMC6226313/>
5. <https://www.nature.com/articles/s41398-021-01362-2https://www.nature.com/articles/s41398-021-01362-2>
6. <https://www.alz.org/alzheimers-dementia/difference-between-dementia-and-alzheimer-shttps://www.alz.org/alzheimers-dementia/difference-between-dementia-and-alzheimer-s>
7. <https://www.phgfoundation.org/blog/dementia-risk-prediction-models/https://www.phgfoundation.org/blog/dementia-risk-prediction-models/>
8. <https://bmccgeriatr.biomedcentral.com/articles/10.1186/s12877-024-05044-8https://bmccgeriatr.biomedcentral.com/articles/10.1186/s12877-024-05044-8>
9. <https://pubmed.ncbi.nlm.nih.gov/36711767/https://pubmed.ncbi.nlm.nih.gov/36711767/>
10. <https://www.nationalelfservice.net/mental-health/dementia/personalised-medicine-dementia/https://www.nationalelfservice.net/mental-health/dementia/personalised-medicine-dementia/>
11. <https://pmc.ncbi.nlm.nih.gov/articles/https://pmc.ncbi.nlm.nih.gov/articles/>

Resources used:

1. Apply map and lambda function for trailing zeros: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.applymap.htmlhttps://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.applymap.html>
2. 'Eval' function: <https://docs.python.org/3/library/functions.html#evalhttps://docs.python.org/3/library/functions.html#eval>
3. Boxplots: https://matplotlib.org/stable/gallery/statistics/boxplot_color.html#sphx-glr-gallery-statistics-boxplot-color-pyhttps://matplotlib.org/stable/gallery/statistics/boxplot_color.html#sphx-glr-gallery-statistics-boxplot-color-py
4. Seaborn: https://seaborn.pydata.org/tutorial/function_overview.htmlhttps://seaborn.pydata.org/tutorial/function_overview.html
5. Z-tests in Python: <https://www.geeksforgeeks.org/z-score-for-outlier-detection-python/https://www.geeksforgeeks.org/z-score-for-outlier-detection-python/>
6. Plotting Piecharts: https://www.w3schools.com/python/matplotlib_pie_charts.asphttps://www.w3schools.com/python/matplotlib_pie_charts.asp
7. Plotting Area plots: <https://matplotlib.org/stable/tutorials/pyplot.htmlhttps://matplotlib.org/stable/tutorials/pyplot.html>
8. Generating Word Cloud: <https://www.analyticsvidhya.com/blog/2021/05/how-to-build-word-cloud-in-python/https://www.analyticsvidhya.com/blog/2021/05/how-to-build-word-cloud-in-python/>
9. Using MatPlotlibs : <https://matplotlib.org/https://matplotlib.org/>